

Q1. Write a Java function to calculate factorial procedurally.

```
public class FactorialProcedural {
    static long factorial(int n) {
        long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    public static void main(String[] args) {
        int n = 6;
        System.out.println("Factorial of " + n + " is " + factorial(n));
    }
}
```

This function is written procedurally: it takes an input value, performs a well-defined sequence of steps (initializing a variable, looping, multiplying) and returns a single result, without relying on any object state.

Q2. Differentiate between procedure and method in Java.

The terms procedure and method are closely related but not identical in Java:

- A procedure, in general programming terminology, is a named block of code that performs a task, usually taking parameters and optionally returning a value; it exists independently of any object.
- In Java, every block of reusable, callable code is technically called a method, because Java has no free-standing procedures outside of a class -- every method belongs to a class.
- A static method in Java behaves like a traditional procedure: it can be called using the class name, without needing an object instance, and typically does not depend on instance-specific state.
- A non-static (instance) method, by contrast, belongs to a specific object and can access and modify that object's fields, aligning with object-oriented behavior rather than plain procedural behavior.

In short, all procedures in Java are implemented as methods, but only static methods closely resemble procedures in the traditional procedural-programming sense, since they operate independently of any particular object.

Q3. Identify drawbacks of procedural programming using a long Java function. Consider the following long procedural function that processes an order end-to-end -- validating input, calculating totals, applying discounts, and printing a receipt, all within a single method.

```
public class OrderProcessor {
    static void processOrder(String customer, double price, int qty,
                             boolean isMember) {
        if (price <= 0 || qty <= 0) {
            System.out.println("Invalid order");
            return;
        }
        double total = price * qty;
        if (isMember) {
            total = total * 0.9;
        }
        double tax = total * 0.05;
        double finalAmount = total + tax;
        System.out.println("Customer: " + customer);
        System.out.println("Subtotal: " + total);
        System.out.println("Tax: " + tax);
        System.out.println("Final Amount: " + finalAmount);
    }

    public static void main(String[] args) {
        processOrder("Anita", 250.0, 3, true);
    }
}
```

This single function reveals several drawbacks of procedural programming when logic is not broken into smaller units:

- **Poor readability:** many unrelated responsibilities (validation, discount calculation, tax calculation, printing) are mixed together in one long block.
- **Difficult maintenance:** changing the tax rate or discount rule requires editing deep inside a large function, risking unintended side effects elsewhere in the same function.
- **Limited reusability:** the discount or tax logic cannot easily be reused elsewhere without copying code, since it is not separated into its own method.
- **Harder testing:** it is difficult to test just the discount calculation or just the tax calculation in isolation, since they are not independent units.
- **No natural way to model the order as a reusable entity with its own state,** unlike an object-oriented design.

These issues are commonly cited motivations for moving toward modular and object-oriented design, where each responsibility is separated into its own method or class.